

ECE 260C, Spring 2026

PlaceOpt & OpenROAD Resizer

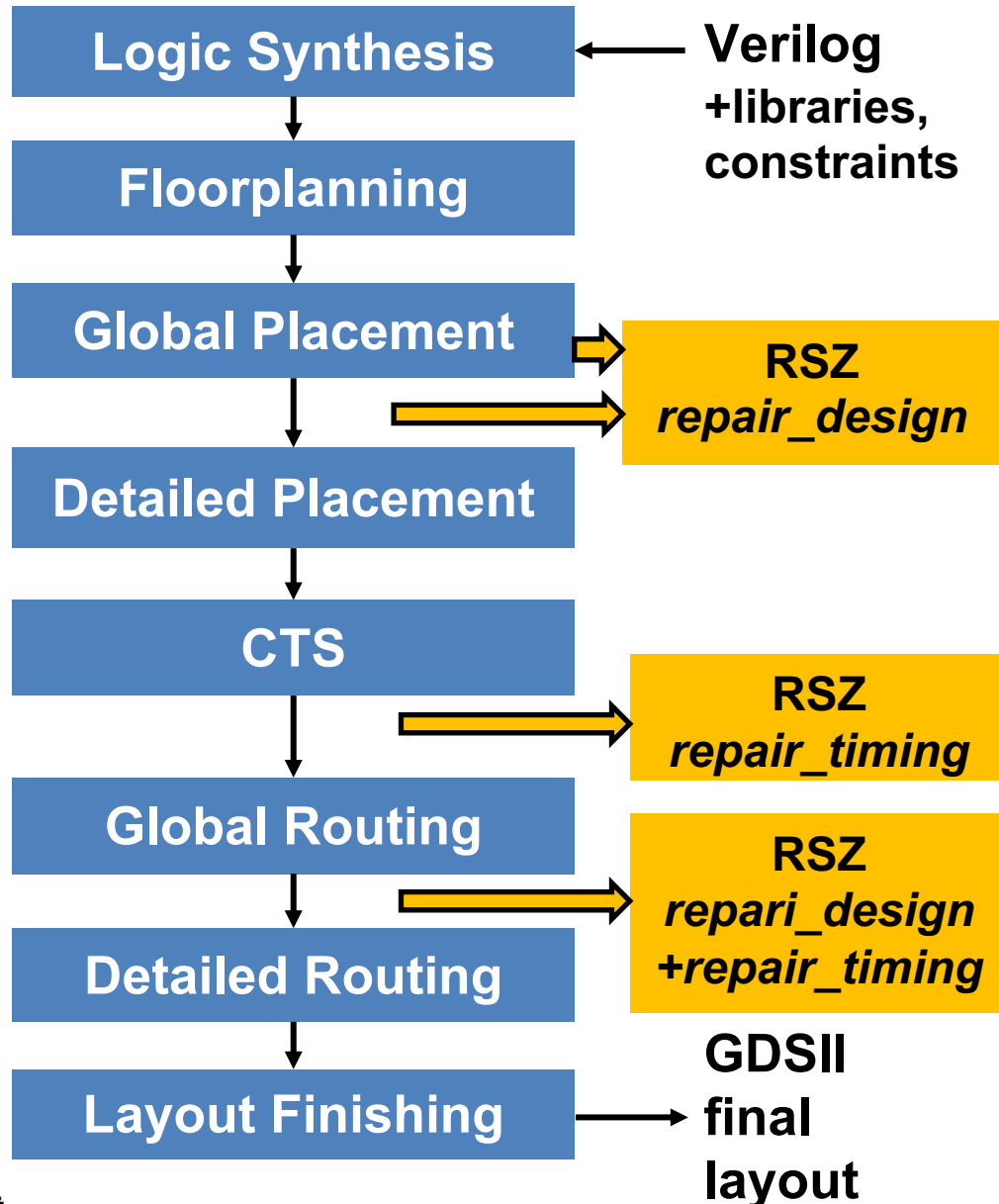
Andrew B. Kahng

Thanks to: Yiting Liu

Agenda

- **Big Picture and Motivation**
- Fundamentals of Buffering and Gate Sizing
- OpenROAD Resizer: Architecture & Code
 - Repair_design
 - Repair_timing
- ML Opportunities and Open Research Questions
- Takeaways

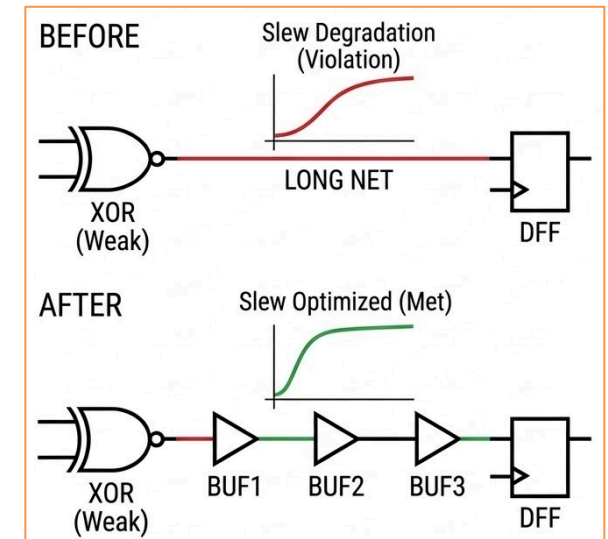
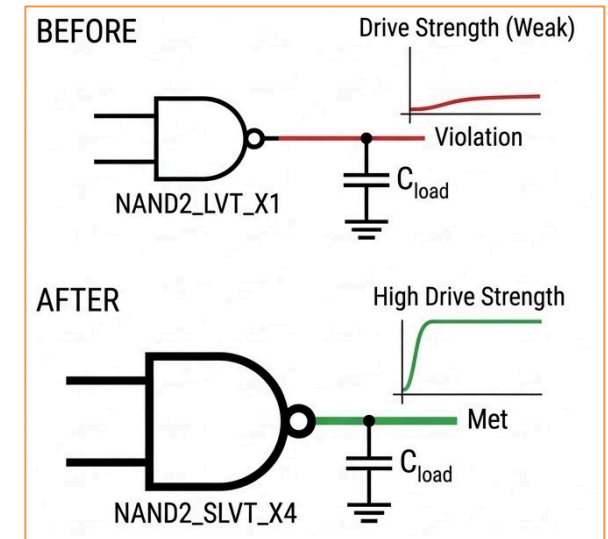
Where Does Resizer Fit in the RTL-to GDSII Flow?



- **Resizer** is invoked *multiple times*
 - *It runs wherever the physical picture becomes clearer*
- **During/post-global placement**
 - `repair_design`: fix ERC violations
 - **Maximum freedom, but parasitics are the least accurate**
- **Post-CTS**
 - `repair_timing`: fix setup and hold violations
 - **Freedom shrinks**. Cannot move clock buffers, resize clock cells, and change clock net topology
- **Post-global-routing**
 - `repair_design+repair_timing`: final timing touch-up and power recovery
 - **Reduced freedom**. Changes should be incremental

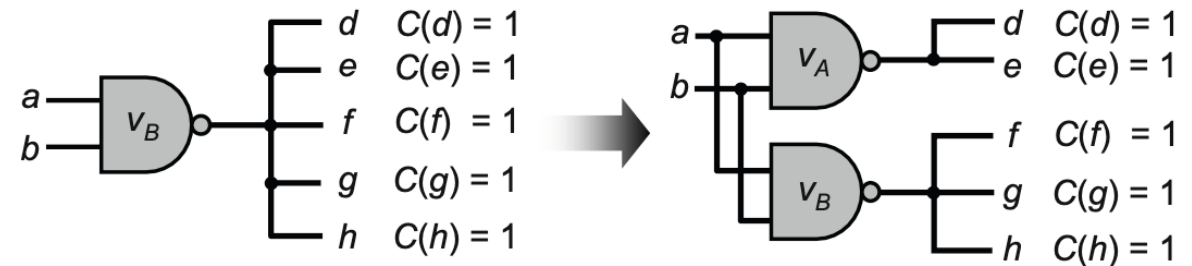
What is “Resizer” ?

- A netlist optimizer that adjusts cells and nets to meet timing, power and electrical constraints
- Core operations
 - **Gate sizing**
 - Replace a cell with a functionally equivalent cell of **different drive strength** from the same Liberty library
 - Larger = **faster** → improve slew and delay
 - **But** larger cell has **higher** input cap → **heavier** load on its driver
 - **Buffering**
 - Insert repeaters (buffers/inverters) on a net
 - **Fix ERC violations** or **break long wires**
 - **But** buffering **increase area and power**
 - **Buffer Removal**
 - Remove unnecessary buffers inserted by previous stages
 - Free up slack for re-optimization



What is “Resizer” ?

- Core operations
 - **Pin swapping**
 - Swap signals among logically equivalent input pins of the same gate
 - Improve **delay** or **transition**
 - Usually a **low-cost optimization** because no new cell is added
 - **VT swapping**
 - Replace a cell with a **functionally equivalent** version that has a different **threshold voltage**
 - Used to trade off delay and leakage power
 - **Lower-VT cells: switch faster, leak more power → used on critical paths**
 - **Higher-VT cells: switch slower, leak less power → used on noncritical paths**
 - **Gate cloning**
 - Duplicate a gate to split its fanout
 - Reduce the load seen by each driver
 - **Improve timing or slew**
 - **But increase area and power**



The resizer must navigate these trade-offs under constraints

Agenda

- Big Picture and Motivation
- **Fundamentals of Buffering and Gate Sizing**
- OpenROAD Resizer
 - Repair_design
 - Repair_timing
- ML Opportunities and Open Research Questions
- Takeaways

Gate Sizing Problem Formulation

- **Given**
 - A placed netlist
 - A Liberty cell library: each logic function has multiple size variants (e.g., NAND2_X1, X2, X4, X8) with different delay, area, input cap, and leakage
- **Choose:** a size variant for each gate
- **Objective:**
 - Fix ERC violations
 - Improve timing
 - Minimize total area and power
- This is a **discrete optimization** problem
 - Each gate has a finite set of candidate cells
- Practical designs have millions of gates, each with 5-20 size variants
→ **enormous search space**

(Gate sizing + Vth-swap) vs. Process Technology

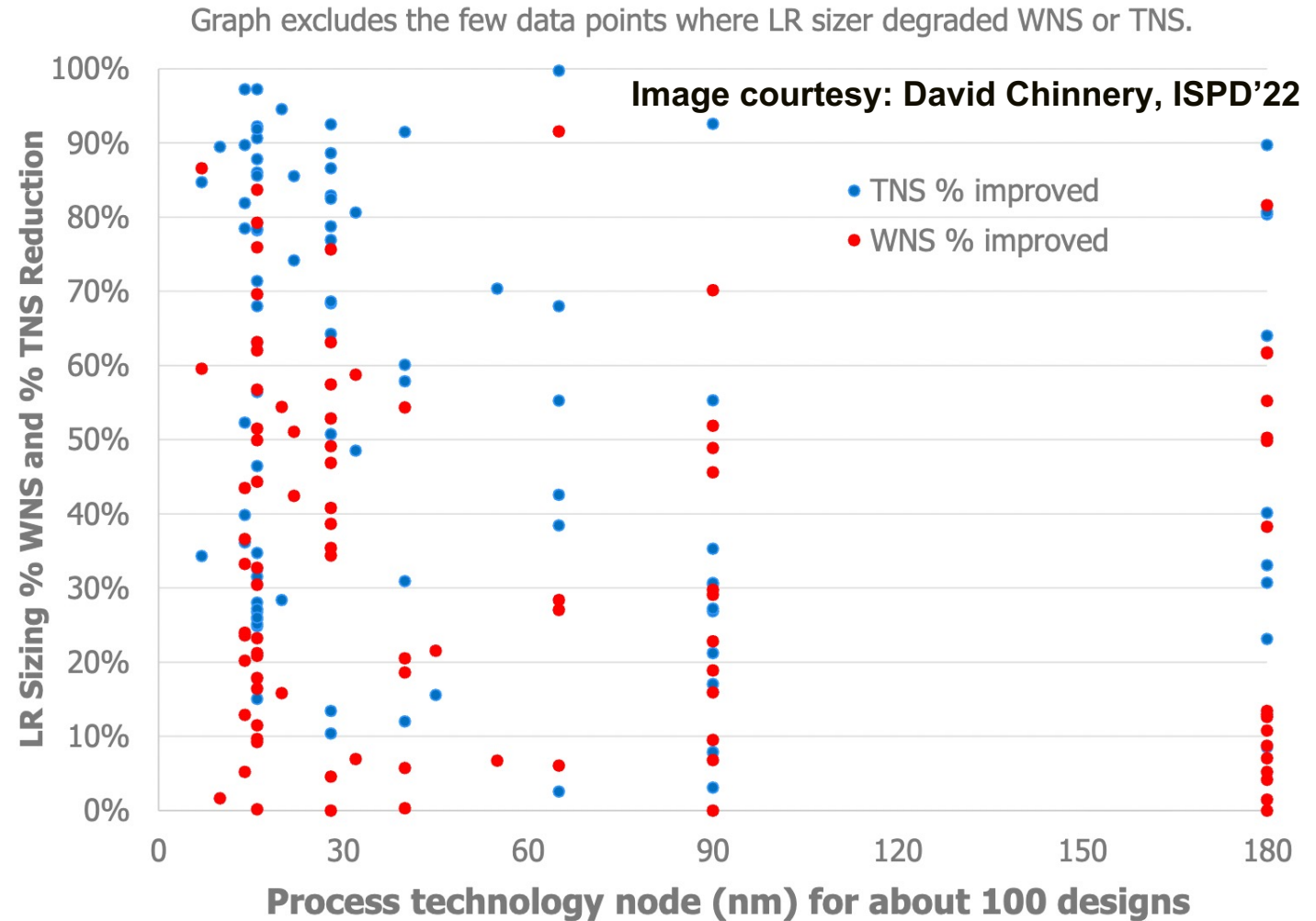
- Gate sizing **can fix** most of outstanding setup timing WNS and TNS

Motivation

- Greedy heuristics are sub-*optimal*
- **Goal: faster + better**
- **Incremental** sizing for WNS, TNS, area, power objectives

Solutions

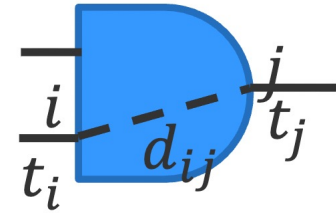
- Lagrangian Relaxation (LR), sensitivity-based, linear programming, dynamic programming, ML, LLMs (?) ← *well studied across literature*



Lagrangian Gate Sizing – a **global** view of the problem

- **Primal** problem is to minimize **normalized** power ($\hat{p}$) and area ($\hat{a}$)

$$\text{minimize}_{\mathbf{x}, \mathbf{t}} \quad \sum_{g \in \text{cells}} \hat{p}_g(x_g) + \hat{a}_g(x_g) \quad \left. \vphantom{\sum_{g \in \text{cells}}} \right\} \text{ Gate sizing objective}$$



$$\text{subject to} \quad \begin{aligned} t_i + d_{ij}(\mathbf{x}) &\leq t_j & \forall (i, j) \in \text{timing arcs} \\ t_k &\leq T_k & \forall k \in \text{timing end points} \end{aligned} \quad \left. \vphantom{\sum_{g \in \text{cells}}} \right\} \text{ Setup timing constraints}$$

$$\text{Max. load cap constraints} \quad \left\{ \begin{aligned} c_{\text{eff},i}(\mathbf{x}) &\leq C_{\text{max},i}(\mathbf{x}) & \forall i \in \text{output timing nodes} \\ s_i(\mathbf{x}) &\leq S_{\text{max},i}(\mathbf{x}) & \forall i \in \text{input timing nodes} \end{aligned} \right\} \text{ Max. slew constraints}$$

- Lagrangian **relaxation (LR)**: replace hard timing constraints with λ **weighted penalty** terms in the objective (**relaxation**) $\rightarrow$ each violation is “priced” rather than “forbidden”
- Example LR **sub-problem (LRS)**: primal objective + **Lagrangian** relaxed timing constraints

$$\text{minimize}_{\mathbf{x}} \quad \sum_{g \in \text{cells}} \hat{p}_g(x_g) + \hat{a}_g(x_g) + \left[\sum_{(i,j) \in \text{arcs}} \lambda_{ij} \times \hat{d}_{ij}(\mathbf{x}) \right] \quad \text{Lagrange multipliers weigh setup time constraints}$$

Thanks to: Bodhisatta Pramanik 9

Lagrangian Relaxation Gate Sizing – overview of the flow

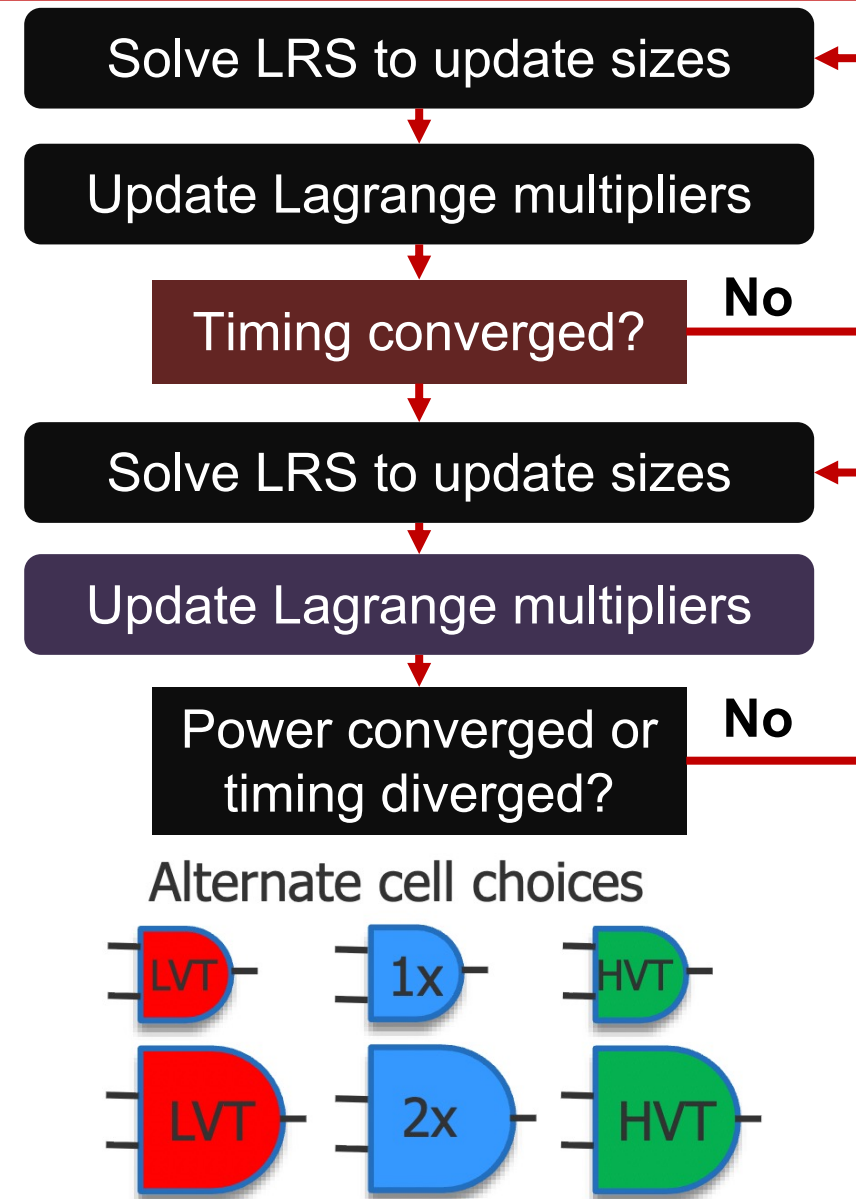
Outer loop: given *sizes of gates*, find (update) *Lagrange multipliers* (λ)

- **Raise** λ on timing arcs with higher violations; **lower** it where slack is positive $\leftarrow$ price each constraint by how much it violates
- Subgradient ascent on $\lambda \leftarrow$ standard update for concave, non-smooth functions
- Project λ on to KKT flow conservation $\leftarrow$ otherwise the **dual** problem is **unbounded** and the sub-problem has no solution

Inner loop: given λ find (update) *gate sizes*

- Solve LRS to resize gates
- **Minimize** the **LRS objective** from the previous slide
- While sizing a gate, check its **local** neighborhood (windows)
- λ values are **fixed** as are for neighboring gate sizes
- Gate isn't resized if it **locally** worsens negative timing slack

Repeat both loops across many “phases” $\rightarrow$ timing recovery phase, power recovery phase, etc ...



Lagrangian Gate Sizing – status, limits, and what comes next

- **Still a major classical backbone** – LR-based gate sizing is still described as the **SOTA** classical approach *← IBM and Siemens EDA have historically implemented LR engines in their tools*
- **Key strengths:** principled timing-aware optimization, scalable, provenance on ISPD2012 and ISPD2013 contests, theoretically sound
- **Key weaknesses:** discrete sub-optimality, convergence is finicky, delay-model sensitive, expensive timing-updates
- **Where the field is moving:** learned and hybrid engines, GNN/RL/differentiable gate sizing, tighter integration of placement/buffering
- **Advent of LLMs:** can potentially be used for outer loop optimization, learning better Lagrangian multipliers, etc.

Buffer Insertion Problem Formulation

- **Given**

- A net connecting a driver to multiple sinks
- A set of buffer types from .lib

- **Choose**

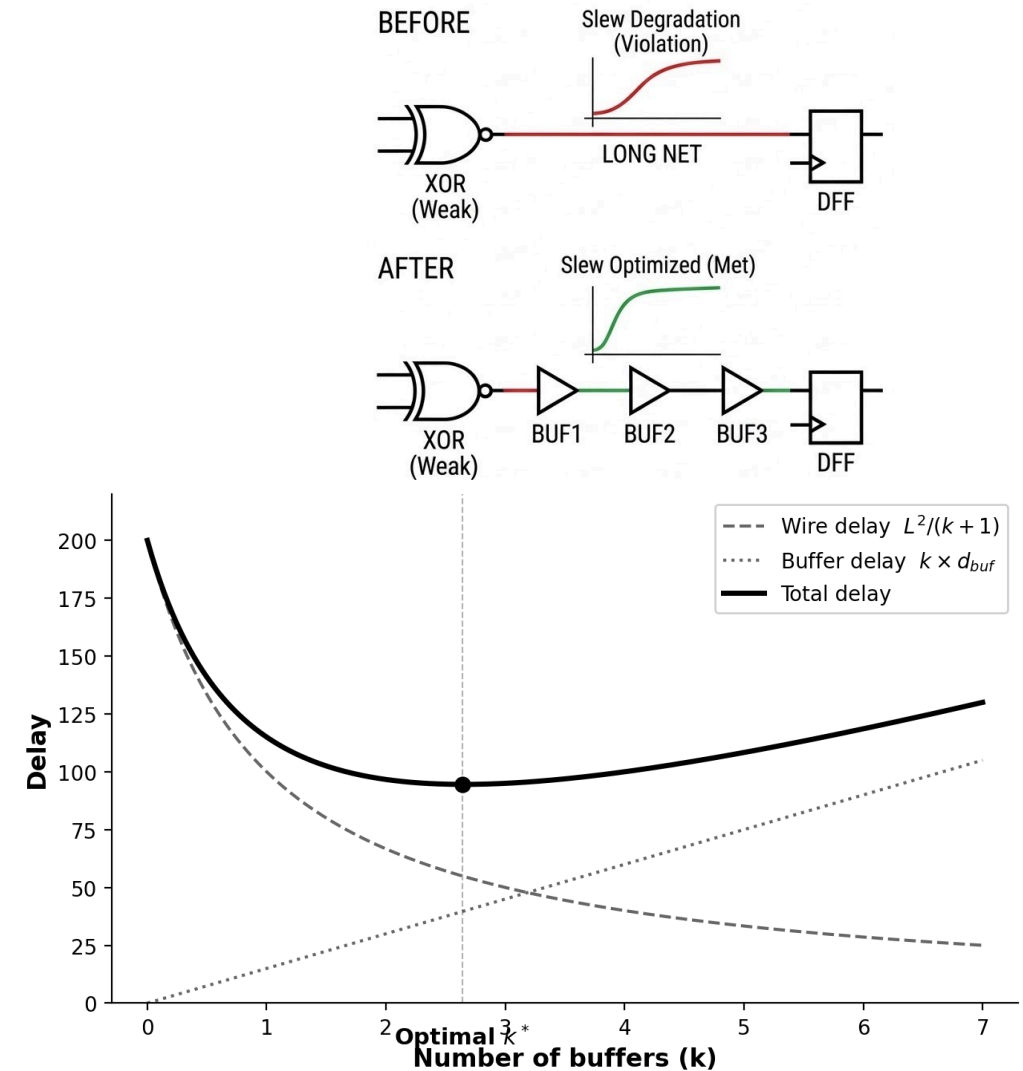
- Where along the net to insert buffers
- Which buffer types at each location

- **Objective**

- Fix ERC violations
- Reduce long wires
- Improve timing
- Minimize total area and power

- **Too few → wire delay dominates**

- **Too many → buffer intrinsic delay dominates**



Van Ginneken's Algorithm

- The **classic algorithm** for optimal buffer insertion on a routing tree
- **Dynamic programming, bottom-up** from sinks to source
- At each node v , maintain a set of **nonredundant candidates** $N(v)$
 - (C, Q) : downstream cap C , and required arrival time Q at this point
- **Three operations** as we traverse bottom-up
 - **Add buffer** at a buffer position
 - **Add wire** through an edge
 - **Merge** two subtrees at a branch point
- **Pruning**
 - Candidate α_1 **dominates** α_2 if $Q(v, \alpha_1) \geq Q(v, \alpha_2)$ **and** $C(v, \alpha_1) \leq C(v, \alpha_2)$
- **Complexity**: $O(n^2)$ time and space
 - n = number of buffer positions



Update C and Q

Agenda

- Big Picture and Motivation
- Fundamentals of Buffering and Gate Sizing
- **OpenROAD Resizer**
 - Repair_design
 - Repair_timing
- ML Opportunities and Open Research Questions
- Takeaways

Main Source Code Structure

- Resizer module in OpenROAD: [OpenROAD/tree/master/src/rsz/src](https://github.com/ucsd-openroad/OpenROAD/tree/master/src/rsz/src)

```
src/rsz/  
├─ src/  
│   ├── Resizer.cc          # Core class  
│   ├── RepairSetup.cc      # Setup timing  
│   ├── RepairHold.cc       # Hold timing  
│   ├── RepairDesign.cc     # ERC repair  
│   ├── Rebuffer.cc         # Rebuffering  
│   ├── BufferedNet.cc      # Buffered net  
│   ├── RecoverPower.cc     # Power opt  
│   └─ ...  
└─ include/rsz/Resizer.hh
```

- Resizer.cc /.hh
 - Main entry point
- RepairDesign.cc
 - repair_design implementation
- RepairSetup.cc
 - Setup timing repair
- RepairHold.cc
 - Hold timing repair
- Rebuffer.cc
 - Rebuffering (van Ginneken-style)
- RecoverPower.cc
 - Power recovery (downsizing with positive slack)

repair_design

- **Fix ERC violations, not timing violations**

- Max capacitance violations
- Max slew violations
- Max fanout violations
- Long wires

- **Algorithm Flow** (src/rsz/src/RepairDesign.cc)

- Estimate parasitics
- For each net, check ERC violations
- For nets with violations
 - Try **sizing up** the driver
 - If that's not enough, **insert buffers** to split the load
- For long wires
 - Insert buffers along the wire (using Steiner tree topology)
- After each modification, **incrementally update timing** via OpenSTA

```
repair_design
  [-max_wire_length max_length]
  [-slew_margin slew_margin]
  [-cap_margin cap_margin]
  [-max_utilization util]
  [-pre_placement]
  [-buffer_gain float_value] (deprecated)
  [-match_cell_footprint]
  [-verbose]
```

- In timing-driven global placement, GPL invokes **repair_design** at certain overflow thresholds

repair_timing (setup repair)

- Fix setup timing violations *repair_timing -setup*

- **Algorithm Flow** ([src/rsz/src/RepairSetup.cc](#))

- Run STA → find the **worst-slack endpoint**
- Walk backward along the critical path from that endpoint
- At each gate on the path, try **a sequence of "moves"**
 - **Resize** (swap to larger drive-strength variant)
 - **Buffer insertion** (on the output net, using rebuffering)
 - **Pin swap** (reorder equivalent input pins)
 - **Gate cloning** (split fanout)
 - **Vt swap** (swap to faster threshold voltage)
- Accept the move if it **improves slack**; reject otherwise
- Repeat (multi-pass) until no more improvement or the iteration limit is hit

Which move gives the **best improvement per unit cost**?

Sensitivity = (Benefit of the move) / (Cost of the move)


$$\text{Sensitivity} = \frac{\Delta \text{Slack}}{\Delta \text{Area}}$$

repair_timing (hold repair)

- Fix hold timing violations *repair_timing -hold*

- Hold violations = path is too fast (data arrives too early relative to clock)
- Fix: add delay to the data path by inserting buffers or delay cells

- **Algorithm Flow** (src/rsz/src/RepairHold.cc)

- Find all **hold-violated endpoints**
- For each, identify the **shortest path**
- **Insert buffer(s)** along the path to add delay
 - **-max_buffer_percent**: limits total buffers inserted (default 20%)
- Re-check that the setup is not broken by the added delay
- By default, it will **not** insert buffers if it causes **setup violations**
 - Unless **-allow_setup_violations** is specified



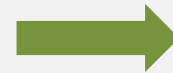
$$\text{Sensitivity} = \frac{\Delta \text{Hold_Slack}}{\Delta \text{Area}}$$

Power recovery

- After setup/hold repair, some paths have **positive slack** (over-designed)
- Conduct cell **downsize** to save power
- ***repair_timing –recover_power***

- **Algorithm Flow** ([src/rsz/src/RecoverPower.cc](#))

- Candidate path selection
 - Collect all timing endpoints with positive setup slack
 - Sort candidates in **descending order** based on the endpoint slacks
- Identify the best gate to downsize (per-path)
 - For each gate, compute **load-dependent delay = arc delay – intrinsic delay**
 - Sort gates by load-dependent delay descending
- Select replacement cell
 - Get all equivalent swappable cells
 - Accept the weakest cell satisfying all constraints
 - **Delay increase fits within slack budget**
 - **No legalization disruption**



$$\text{Sensitivity} = \frac{\Delta \text{Power}}{\Delta \text{Delay}}$$

Incremental STA

- **Problem**

- Full STA traverses the **entire timing graph** (forward + backward): $O(|V| + |E|)$ per run
- The resizer makes thousands of trial moves (resize, insert buffer, pin swap, ...)
- Running full STA after every trial move would be prohibitively **slow**

- How **incremental STA** works in OR (via OpenSTA) ?

- **Forward propagation** (arrival times)

- Arrival-time updates propagate **forward** through the affected **transitive fanout region**

- **Backward propagation** (required times)

- Required-time updates propagate **backward** from affected constrained vertices/endpoints
- The update travels through the corresponding **transitive fanin region**

- **Bounded incremental update**

- OpenSTA can **stop** incremental delay propagation when the **delay change falls below** a user-defined tolerance

- **Lazy evaluation**

- Arrival, required, and slack are recomputed only when needed

Each optimization move modifies only a small set of nets

Recent Improvements to Resizer

- **Smarter optimization moves (-sequence)**
 - Configurable move sequence: **unbuffer** → **vt_swap** → **sizeup** → **swap** → **buffer** → **clone** → **split**
 - **Ordering** matters: cheap moves (Vt swap, unbuffer) are tried globally before expensive moves
 - **Vt swap** is now applied earlier and more systematically
- **Better candidate selection**
 - **Buffer filtering** by Vt type, cell footprint, drive strength, and site compatibility before rebuffering
 - **-limit_sizing_area / -limit_sizing_leakage**: prevent overly aggressive moves
 - **-keep_sizing_site / -keep_sizing_vt**: restrict swaps to cells with the same physical site or threshold voltage class
 - Early sizing/buffering cap ratio heuristics (**set_opt_config**) for pre-CTS stages where parasitics are uncertain
- **Phase-based setup repair**
 - **repair_timing -setup** now exposes **phase control** via **-phases**
 - **Phase 1 (WNS-focused)**: fix the worst endpoint first (WNS-focused)
 - **Phase 2 (TNS-focused)**: scan violating endpoints from worst to least-violated to eliminate the long tail of violations
 - **Phase 3 (Cone-based)**: target shared upstream logic so one move helps multiple endpoints
 - **Phase 4 (Last-gasp)**: apply more aggressive final fixes on the hardest remaining violations

Typical Usage and TCL Commands

Typical RSZ command sequence

```
read_sdc design.sdc
set_wire_rc -layer metal2
set_dont_use {CLKBUF_*}
```

Post-placement

```
estimate_parasitics -placement
repair_design
detailed_placement
```

Post-CTS

```
clock_tree_synthesis
estimate_parasitics -placement
repair_timing
detailed_placement
```

Post-global routing

```
global_route
estimate_parasitics -global_routing
```

```
repair_design
global_route -start_incremental
detailed_placement
global_route -end_incremental
```

```
estimate_parasitics -global_routing
repair_timing ← fix setup/hold
global_route -start_incremental
detailed_placement
global_route -end_incremental
```

Agenda

- Big Picture and Motivation
- Fundamentals of Buffering and Gate Sizing
- OpenROAD Resizer
 - Repair_design
 - Repair_timing
- **ML Opportunities and Open Research Questions**
- Takeaways

ML Opportunity 1: ML for Buffering

- **Motivation**

- Timing closure requires extensive buffer insertion
 - Buffer insertion during the global placement process
 - Fix ERC violations
 - Give more accurate slack information
 - Consider the buffer area effect
 - Actual buffer insertion slows down the GPU-accelerated placer
because the topology changes!
- ➡ **Buffer-porosity-aware placement**
- ➡ **Lightweight virtual buffering strategy**

- **Limitations of existing methods**

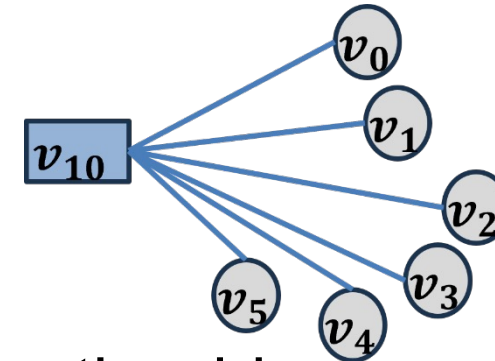
- Traditional van Ginneken-Lillis-style methods
 - Computationally expensive
- Modern ML-based approaches
 - Omit Electrical Rule Check (ERC) considerations
 - **Fail to “close the loop”** back into the physical design flow

- A recursive learning-based generative buffering model
 - Predicts buffer locations
 - Predicts buffer types
 - Predicts buffered tree structures
- **MLBuf is integrated into RePIAce: MLBuf-RePIAce**
 - An **open-source** learning-driven virtual buffering-aware analytical global placement framework
 - Operates within a **complete placement and optimization flow** (closing the loop)
 - Achieve better post-route PPA results in both OpenROAD flow and commercial flow

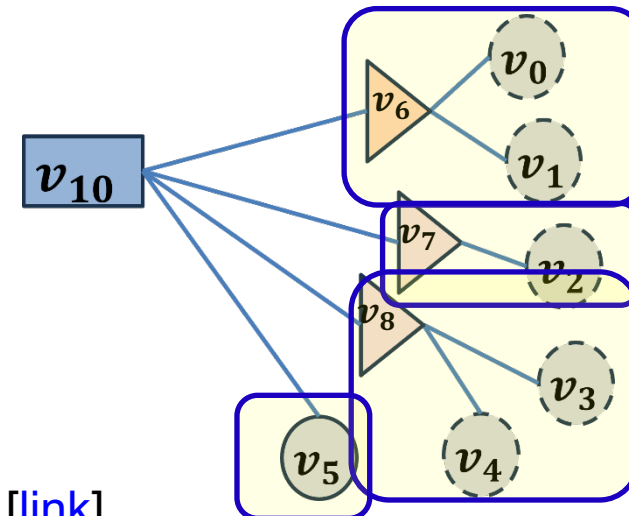
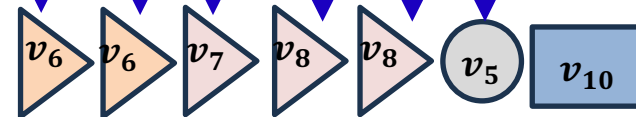
[1] A. B. Kahng, Y. Liu, and Z. Wang, "[Recursive Learning-Based Virtual Buffering for Analytical Global Placement](#)", Proc. MLCAD, 2025 (Best Artifact Award)

Reformulation of Buffer Insertion

- Model buffer insertion as **sequence transformation**
- Consider each net independently
- Represent cells within the same net as a **sequence**
- **Input:** A sequence of a driver and its sinks



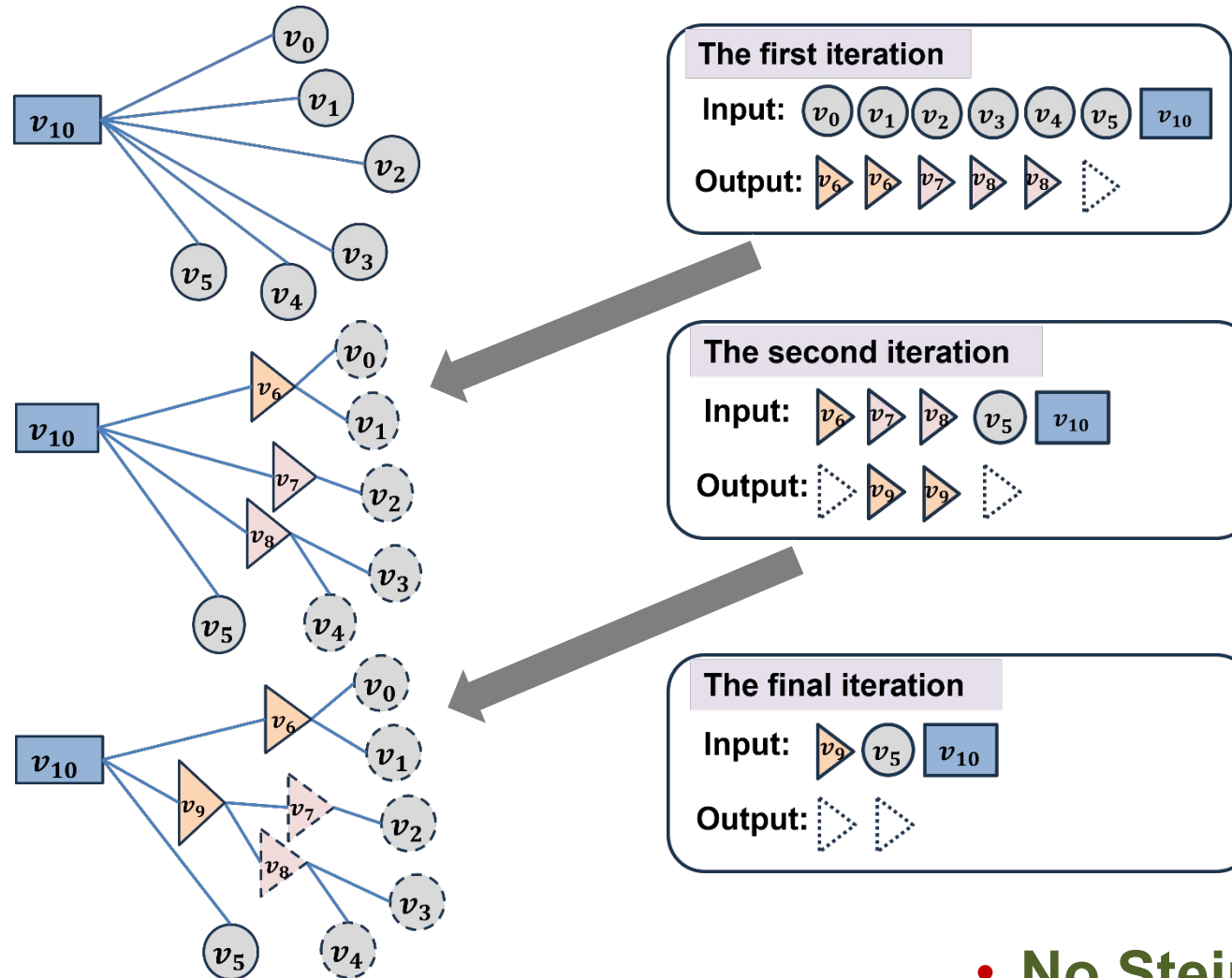
- **Output:** A transformed sequence containing the driver, predicted buffers, and unbuffered sinks



BufFormer [\[link\]](#)

Reformulation of Buffer Insertion

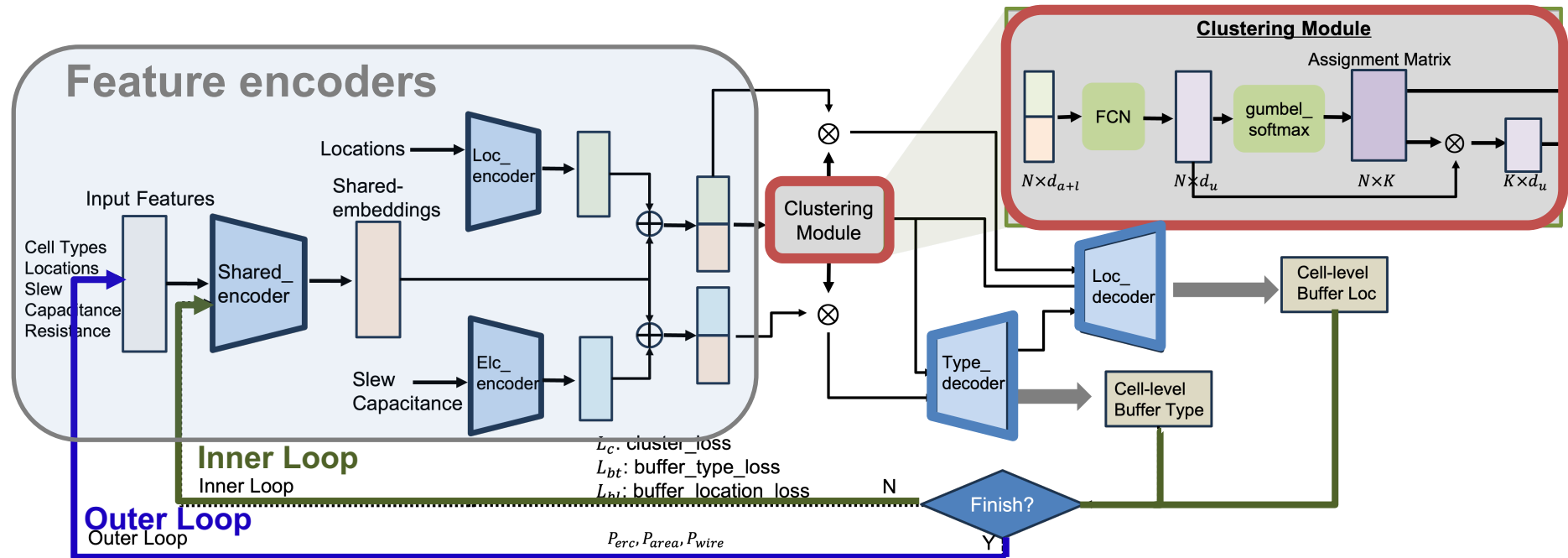
- MLBuf mimics the bottom-up dynamic programming



- No Steiner tree construction**

ML-based Virtual Buffering (MLBuf)

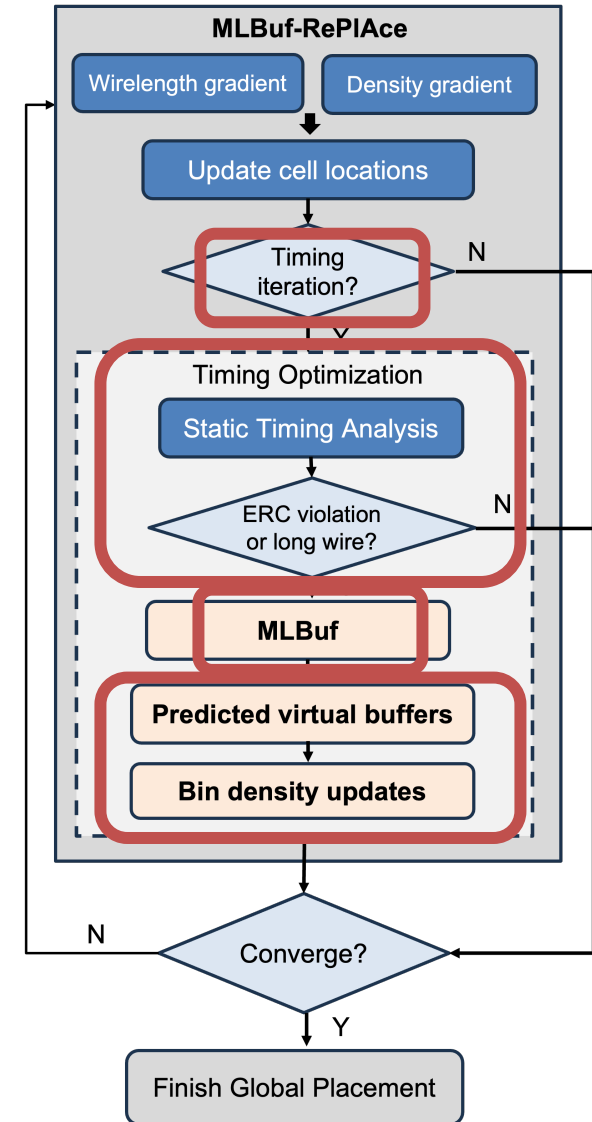
- Feature **encoders**: extract spatial and electrical features
- **Differentiable clustering module**: end-to-end training
- Task-specific **decoders**: output buffer types and locations
- Dual-level optimization flow: **inner-loop** loss functions and **outer-loop** global penalties



MLBuf-RePIAce: Buffer Porosity-Awareness

- **Trigger:** Overflow > threshold
- **Timing analysis:** OpenSTA finds problem nets
- **MLBuf:** Feed the problem nets to pre-trained MLBuf model
repair_design focus on ERC fixing
- **Virtual buffering:** → virtual area occupancy in placement bins

The placer **reserves space**, reducing routing **congestion** and improving post-route PPA



Experimental Results

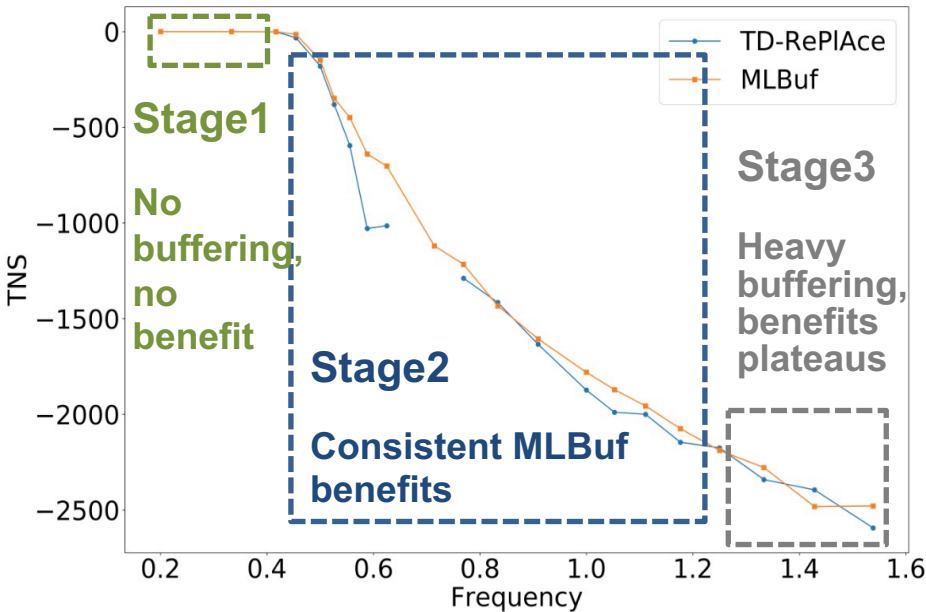
- PPA validation with OpenROAD

Design	Method	Metrics			
		rWL	WNS	TNS	Power
ibex	RePIAce	303209	-0.301	-22.750	128.661
	TD-RePIAce	299212	-0.162	-15.361	127.828
	Ad-Hoc	298973	-0.295	-30.389	128.876
	MLBuf	299049	-0.234	-12.112	127.343
jpeg	RePIAce	630755	-0.105	-4.669	551.357
	TD-RePIAce	634789	-0.146	-5.651	559.573
	Ad-Hoc	637868	-0.100	-3.611	573.129
	MLBuf	640555	-0.082	-2.524	567.337

- PPA results with Commercial tool

Design	Method	Metrics			
		rWL	WNS	TNS	Power
jpeg	RePIAce	588917	-0.115	-85.659	536.972
	TD-RePIAce	588099	-0.089	-68.505	547.551
	Ad-Hoc	588917	-0.115	-85.659	536.972
	MLBuf	582866	-0.068	-46.013	535.932
ariane	RePIAce	4781145	-0.169	-157.028	846.221
	TD-RePIAce	4805300	-0.134	-104.777	846.393
	Ad-Hoc	4781145	-0.169	-157.028	846.221
	MLBuf	4795461	-0.106	-67.962	844.090
BP	RePIAce	27306546	-0.057	-34.223	4188.776
	TD-RePIAce	27373470	-0.067	-40.861	4194.181
	Ad-Hoc	27379181	-0.041	-35.763	4189.003
	MLBuf	27373073	-0.038	-19.246	4188.754

- TCP sweep



- RePIAce: normal RePIAce without timing-driven mode
- TD-RePIAce: default virtual buffering-based timing-driven RePIAce in OpenROAD
- Ad-Hoc: a baseline for comparison

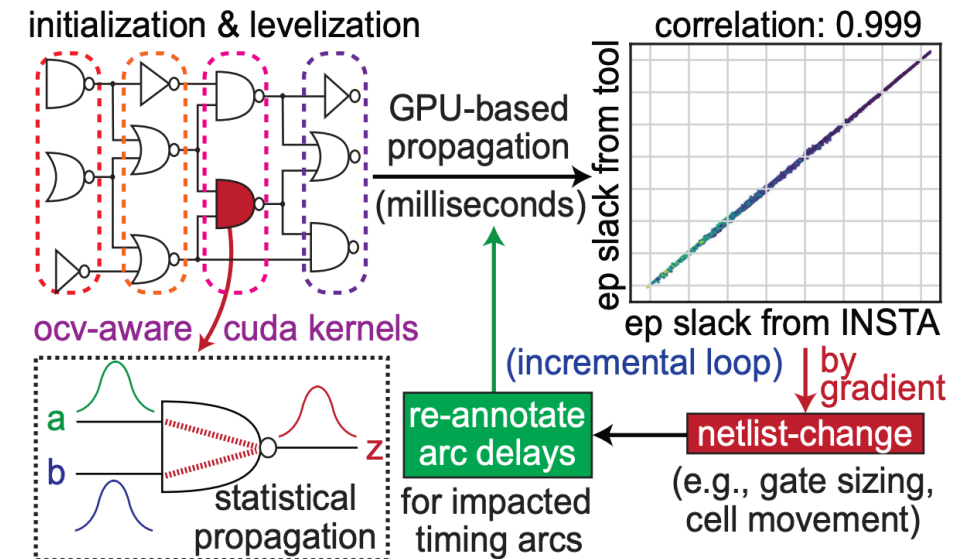
ML Opportunity 2: Fast Timing Prediction as STA Proxy

• Problem

- STA is the **runtime bottleneck** in timing-driven optimization
- Traditional STA tools are **sequential** and **CPU-bound**
- Each trial move triggers an incremental timing update
- For large-scale designs, thousands of STA calls dominate the runtime

• INSTA [2]

- A **fully differentiable** STA engine running on **GPU**
- Begins with a **one-time initialization** from a reference timing engine
- Performs fast, differentiable, and tool-accurate STA propagation
- Enables **backpropagation** of timing gradients through the STA computation graph



[2] Y.-C. Lu, Z. Guo, K. Kunal, R. Liang, and H. Ren, “[INSTA](#): An Ultra-Fast, Differentiable, Statistical Static Timing Analysis Engine for Industrial Physical Design Applications”, *Proc. DAC*, 2025. (Best Paper)

ML Opportunity 3: ML for Sizing

- **Problem**

- Given a gate and its context, predict the optimized size variant directly

- **Representative papers**

- **RL-Sizer** [3]

- **GNN-powered** policy gradient **RL** framework
 - State = circuit graph (encoded by GNN)
 - Action = select a gate and choose a new size
 - Reward = improvement in timing metrics (WNS/TNS)

- **TransSizer** [4]

- **Transformer** architecture for gate sizing
 - Use self-attention to capture global dependencies across gates
 - Directly predict optimal gate sizes for all gates in one forward pass

- **LearnSize** [5]

- **Multi-modal modeling** combining graph features and layout features
 - Efficient gradient generation and back-propagation designed for large-scale circuits
 - Physically-aware: considers placement constraints during sizing

[3] Y. -C. Lu, et al., "RL-Sizer: [VLSI Gate Sizing for Timing Optimization using Deep Reinforcement Learning](#)," Proc. DAC, 2021.

[4] S. Nath, et al., "[TransSizer: A Novel Transformer-Based Fast Gate Sizer](#)," Proc. ICCAD, 2022.

[5] Y. Ye et al., "[Learning-Driven Physically Aware Large-Scale Circuit Gate Sizing](#)," TCAD, 44(5)(2025), pp. 1901-1914.

ML Opportunity 4: GPU-Accelerated & Differentiable Sizing

- Move from discrete greedy heuristics → **continuous, differentiable, GPU-parallelizable** formulations
- **Why differentiable?**
 - If we can compute gradients of timing w.r.t. gate sizes, we can optimize all gates simultaneously
 - GPU parallelism enables evaluating millions of gates in parallel → much **faster**
- **Representative papers**
 - **LEGO-Size** [6]
 - **LLM-Enhanced** GPU-Optimized signoff-accurate differentiable gate sizing
 - **Fusion of global placement and gate sizing** [7]
 - Fuse placement and sizing into a single differentiable optimization
 - **1st place winner of ICCAD 2024 Contest Problem C**

[6] Y. -C. Lu, etc., "LEGO-Size: [LLM-Enhanced GPU-Optimized Signoff-Accurate Differentiable VLSI Gate Sizing in Advanced Nodes](#)," Proc. ISPD, 2025 (Best Paper Award).

[7] Y. Du, etc., " [Fusion of Global Placement and Gate Sizing with Differentiable Optimization](#)," Proc. ICCAD, 2024.

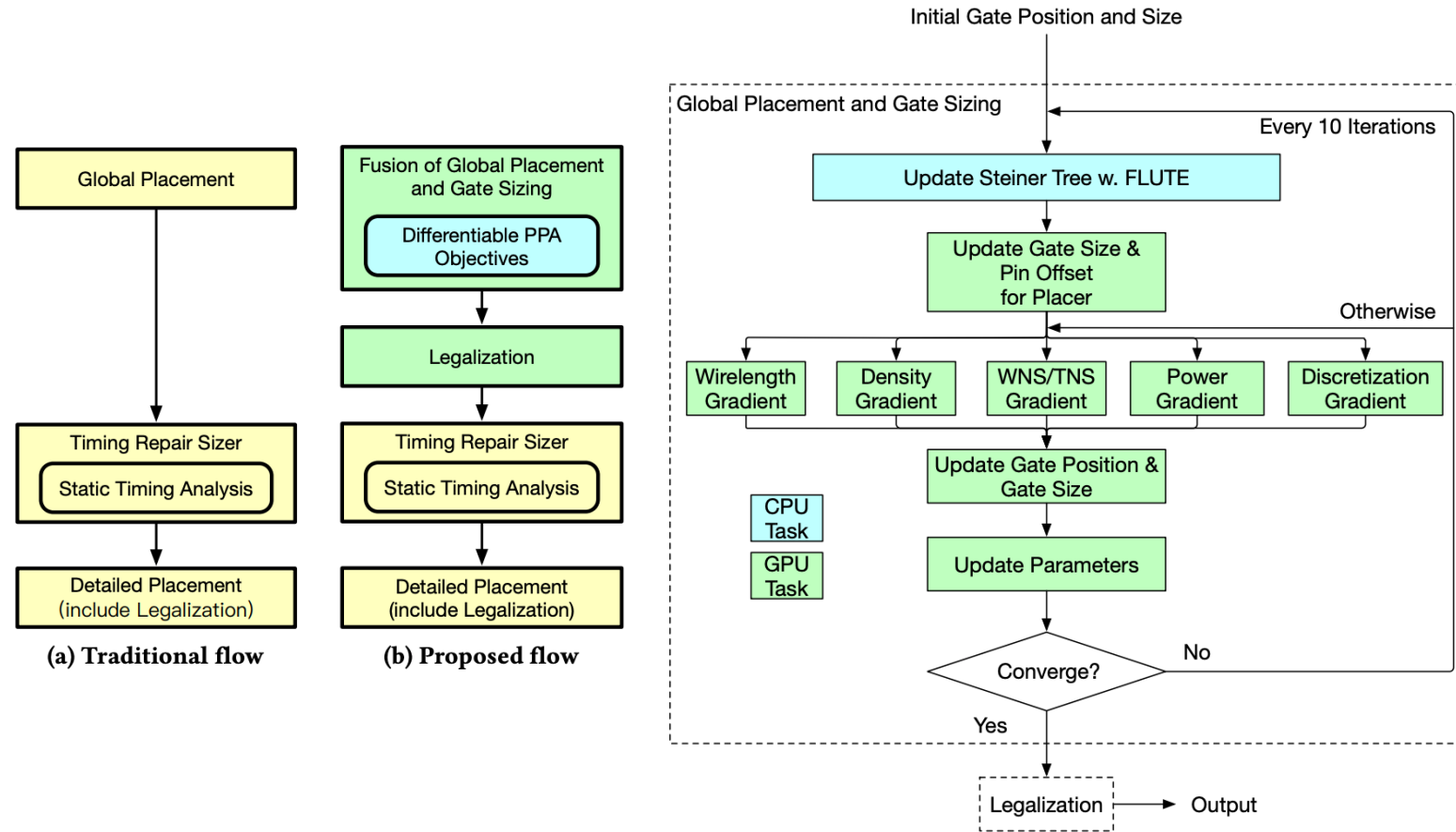
Contests

- **ICCAD 2024 Contest Problem C: Scalable Logic Gate Sizing** [\[link\]](#)
 - Focused on scalable **gate sizing**
 - No buffer insertion, netlist edits, or cell relocation
- **ICCAD 2025 Contest Problem C: Incremental Placement Optimization Beyond Detailed Placement** [\[link\]](#)
 - Post-detailed placement **buffering and sizing**
 - Similar setting to ISPD 2026
 - But simplified testcases, without blockage constraints and post-route evaluation
- **ISPD 2026 Contest: Post-Placement Buffering and Sizing** [\[link\]](#)
 - Post-detailed placement **buffering and sizing**
 - Fixed macros and I/Os
 - **Soft placement blockages**

ICCAD 2024 Contest – Winning Team Solution

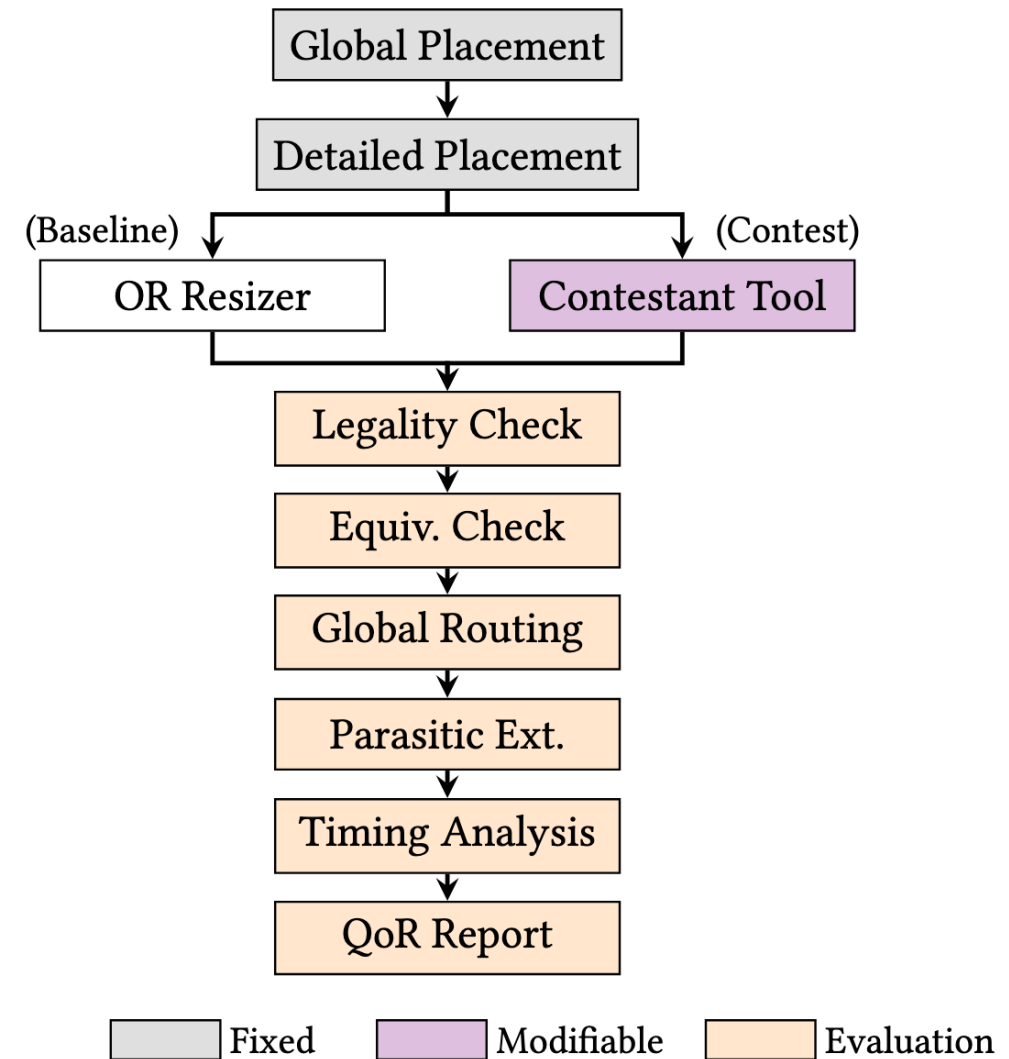
- Fusion of Global Placement and Gate Sizing with Differentiable Optimization

- Both cell locations and gate sizes are **continuous variables** optimized jointly via **gradient descent**
- Treat the gate size as a continuous variable during optimization
- Add an **L1 regularization** term that pushes the gate size toward the nearest discrete library choice
- Uses **differentiable** timing and density models

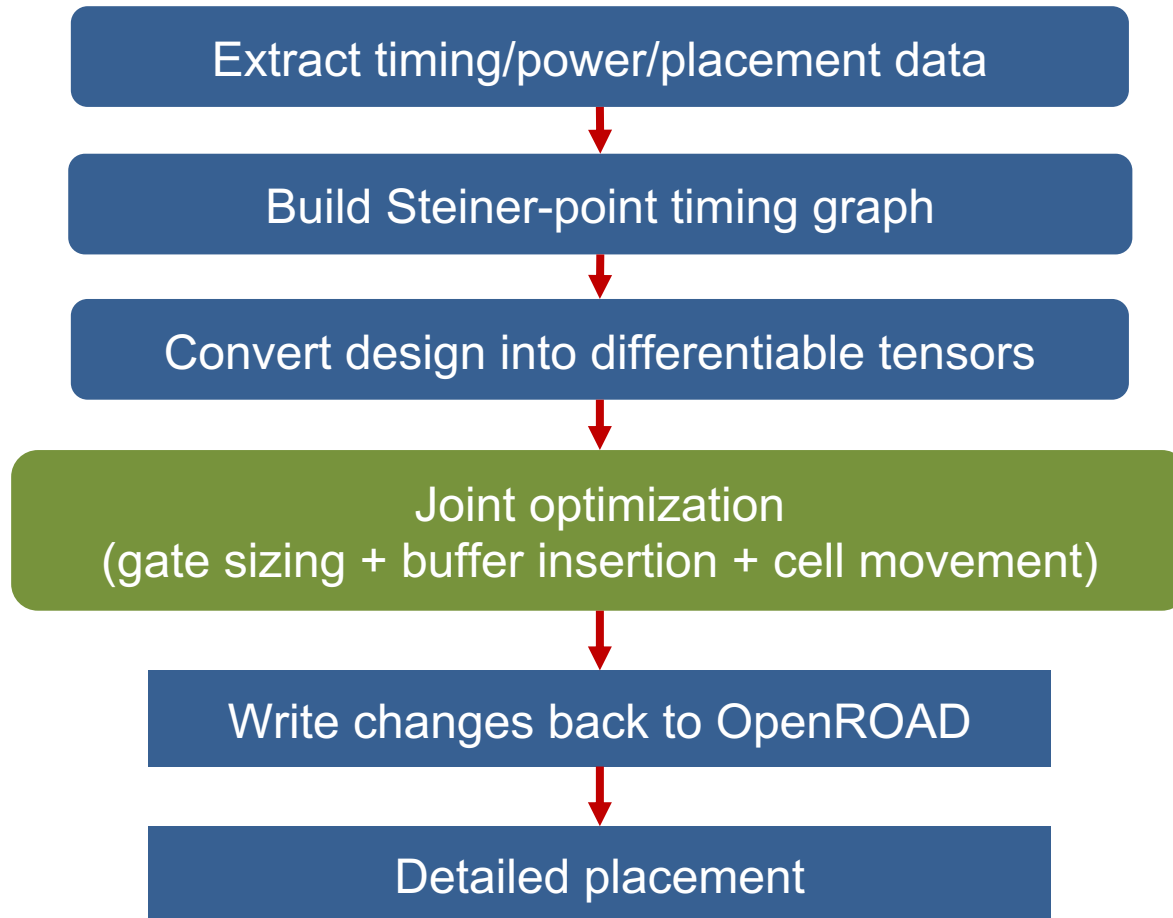


ISPD 2026 Contest

- **Inputs** to contestants:
 - Post-placement DEF
 - Debuffered Verilog netlist
 - .sdc constraints
 - **ASAP7** .lef + .lib
- **Output:** modified netlist + DEF
- **Evaluation:** OpenROAD infrastructure
- **Custom equivalence check** verifies:
 - Functional equivalence between original and resized
 - Instance insertions limited to buffers and inverters
 - Equivalent pin-to-pin connectivity
 - **Buffer tree parity:** even inverter count per source-to-sink path



ISPD 2026 Contest – Winning Team Solution



- **Start from a legalized placement solution**
- **Build a differentiable timing model (PyTorch)**
 - Nets → Steiner trees
 - Cells → equivalent-size candidates
 - Model timing, power, and density
- **Optimize three decisions together**
 - Gate sizing & buffer insertion & placement adjustment
- **Multi-objective optimization**
 - Improve TNS
 - Reduce dynamic and leakage power
 - Penalize overflow/movement
 - Gradually strengthen density and discreteness
- **Commit results back to OpenROAD**
 - Pick the best cell size per gate
 - Insert selected buffers into the netlist
 - Update cell locations
- **Run detailed placement to legalize**

Challenges and Open Research Questions

- **Generalization**
 - Models trained on one technology/design may not transfer to others
 - Few-shot/domain adaption needed
- **Accuracy vs. speed**
 - ML proxies for STA may introduce errors
 - How much approximation is acceptable?
- **Integration**
 - How to embed ML models inside the resizer loop without disrupting the flow?
 - Need **inference < incremental STA time** to be worthwhile
- **Scalability**
 - Real designs have millions of gates.
 - Model scalability on large designs is still a challenge
- **Co-optimization**
 - Sizing, buffering, placement, and routing are deeply coupled
 - Joint optimization is the frontier

Agenda

- Big Picture and Motivation
- Fundamentals of Buffering and Gate Sizing
- OpenROAD Resizer
 - Repair_design
 - Repair_timing
- ML Opportunities and Open Research Questions
- **Takeaways**

Takeaways

- **Resizer** adjusts cells and nets to meet timing, power, and electrical constraints
- **OpenROAD's rsz module implements**
 - repair_design (electrical violations)
 - repair_timing (setup/hold)
 - power recovery
- **ML opportunities are accelerating**
 - GNN-based timing prediction
 - RL for sizing
 - Transformer-based buffer prediction
 - GPU-accelerated differentiable optimization
- **Active contests**
 - ICCAD 2024 (ML gate sizing)
 - ICCAD 2025 (simultaneous sizing + buffering + relocation)
 - ISPD 2026 (post-placement buffering & sizing)

BACKUP
